

개인 Wiki를 웹으로 연결하기

Hermes API Gateway · 작업 경로 · 신뢰 경계

파일 기반 지식 저장소를 안전한 웹 서비스로 연결하는 설계 수업

읽기 → 실행 → 게시를 구분하고, 각 단계의 증거를 확인한다.

2026-10-01 기술문서 기준 · 서비스 구현물이 아닌 강의 자료

이 수업을 마치면

구성요소 이름보다 “누가 무엇을 허용하는가”를 설명할 수 있어야 합니다.

구조를 그린다

Browser, BFF, Gateway, Agent, Wiki의 책임과 신뢰 경계를 구분한다.

계약을 읽는다

cwd, session, run, SSE, revision, receipt가 보장하는 것과 못 하는 것을 설명한다.

오류를 설계한다

연결 단절·중복 요청·중지 경합·불완전 게시에서 안전한 다음 동작을 고른다.

권장 선수 지식: HTTP 요청/응답, JSON, 파일 경로 · 용어는 본문에서 다시 정의

강의 지도

기능을 붙이는 순서가 아니라, 경계를 이해하는 순서로 진행합니다.

학습 단위	핵심 질문
01-02 안내·아키텍처	어디까지가 웹이고 어디부터가 에이전트인가?
03-04 작업 경로·데이터	시작 위치와 권한, 파일과 공개 응답은 어떻게 다른가?
05-06 API·스트리밍	요청 접수, 진행, 재연결, 종료를 어떻게 판정하는가?
07-08 화면·보안	사용자에게 무엇을 보여주고 어떤 권한을 차단하는가?
09-10 게시·실습	언제 저장 완료라 말할 수 있고, 어떤 순서로 검증할까?

증거의 강도를 섞지 않는다

“문서에 있다”와 “실제로 동작했다”는 다른 문장입니다.

표현	뜻	이 강의에서의 취급
관측	당시 파일·출력·소스에서 확인	과거 관측으로 시점을 붙임
공식 계약	조회 당시 공식 문서의 설명	설치 구현과 차이가 날 수 있음
제안	아직 만들지 않은 BFF·UI 설계	현재 endpoint로 말하지 않음
미확인	live API·모델·배포 등 미시험	성공/실패를 추정하지 않음

강의용 예시와 설계 실습은 실제 요청·응답 또는 운영 실행 결과가 아닙니다.

이번 서비스의 출발점

개인용 인증 서비스의 read-only 탐색이 먼저입니다.

초기 목표

- 목록과 기존 노트 탐색
- 출처·검토 상태 확인
- 모델 없이 정형 자료 조회

별도 승인 또는 비목표

- 통제된 질의는 후속 단계
- 컴파일·게시·Cron 제어는 분리
- 공개 무인 에이전트·새 DB/RAG는 초기 범위 아님

웹 화면을 만드는 일이 원문 처리나 자동화 권한을 늘려 주지 않습니다.

수치에는 시각과 단위를 붙인다

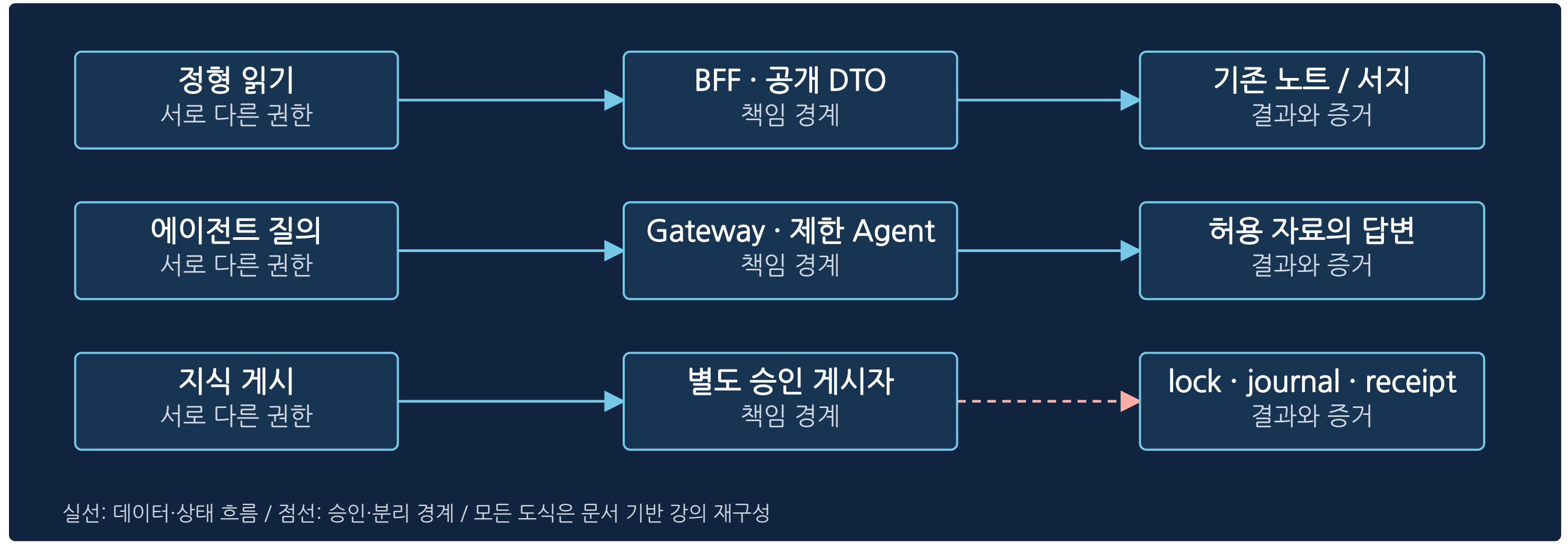
다음은 원문 문서의 과거 snapshot이지, 현재 운영 상태가 아닙니다.

당시 관측	값	해석
01:43:55 UTC 원문 보관	20개 버전: HTML 18 / PDF 2	논문 본문 분석 완료 수가 아님
활성 지식 문서	6개: 논문 2 / 개념 3 / 비교 1	논문별 컴파일 수와 다름
초기 컴파일 ledger	published_draft 2 / blocked_approval 16 / blocked_policy 2	PDF 보관 실패를 뜻하지 않음
02:01:11 UTC v3 전이	blocked_approval 16 / queued 2 / published_draft 2	queued는 완료가 아님

날짜: 2026-10-01 · 형식만 보고 현재 정책 상태를 하드코딩하지 않는다.

하나의 Wiki, 서로 다른 세 경로

읽기·질의·게시를 같은 HTTP 통로로 섞지 않습니다.



정형 조회에는 LLM이 필요 없고, 질의 완료에는 게시 권한이 포함되지 않는다.

BFF는 브라우저를 위한 제한된 서버

Backend for Frontend: 브라우저에 필요한 계약만 제공하는 중간 계층입니다.

인증과 소유권

사용자를 확인하고 page/run/session 접근 권한을 매 요청마다 검사한다.

Projection과 요청 구성

공개 필드만 고른 DTO를 반환하고 서버가 허용한 입력만 upstream으로 보낸다.

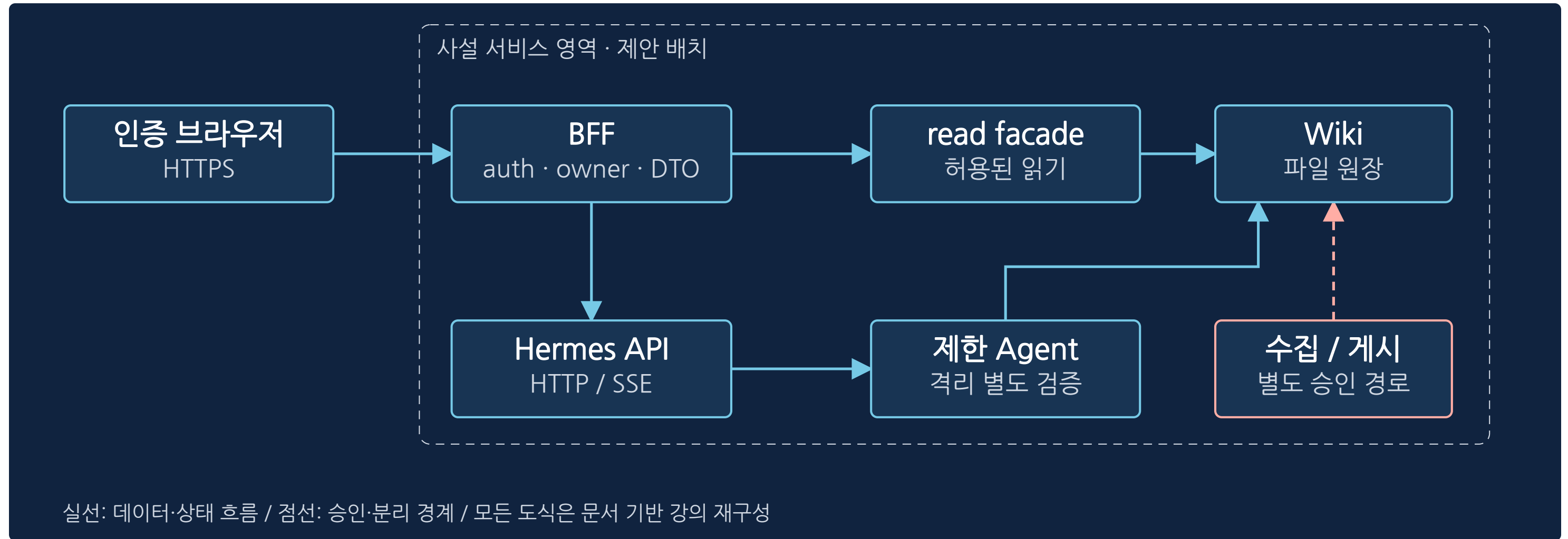
할 수 없는 일

BFF의 API 제한만으로 Agent의 host 파일·도구 접근을 격리할 수는 없다.

BFF ≠ 무제한 reverse proxy · DTO = 외부에 전달하도록 고른 데이터 구조

신뢰 경계가 보이는 전체 구조

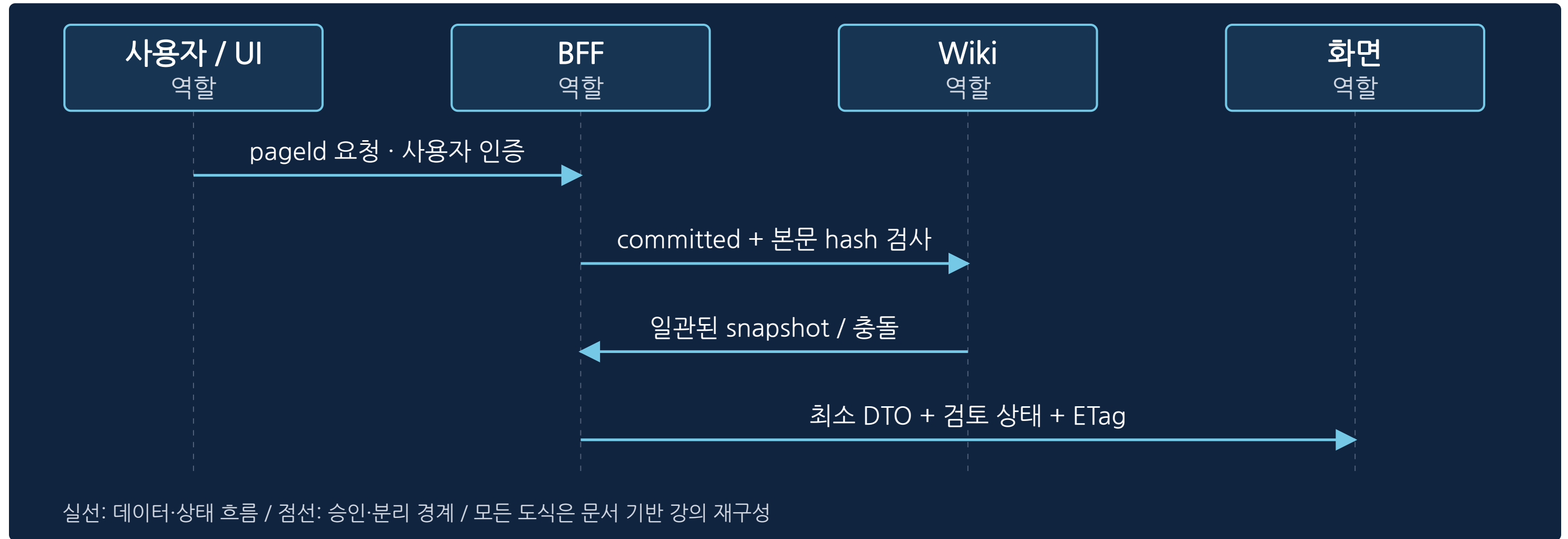
도형은 책임과 제안 배치를 나타냅니다. 서비스 설치 증거가 아닙니다.



인터넷에 노출하는 것은 인증 경계이지 Wiki root나 Gateway 관리 API 전체가 아닙니다.

정형 읽기: 모델 없이 페이지를 연다

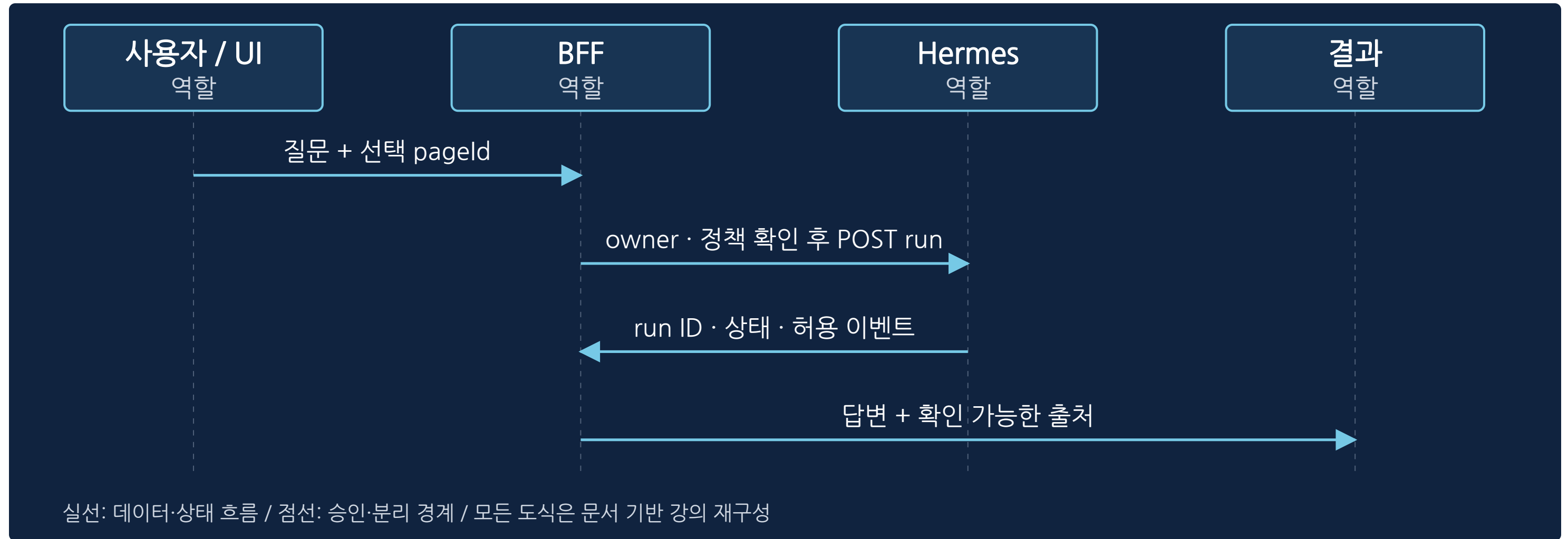
파일 존재 여부가 아니라 일관된 snapshot을 서비스합니다.



읽는 도중 게시가 바뀌면 snapshot_conflict · 빈 페이지나 완료본으로 위장하지 않는다.

질의: 실행 문맥은 BFF가 소유한다

사용자의 질문은 입력이지만, 도구·모델·자료 범위를 정하는 권한은 아닙니다.



답변에는 확인 가능한 page revision/hash와 읽은 범위를 연결한다. 없는 인용은 만들지 않는다.

같은 IP 표기도 다른 위치일 수 있다

127.0.0.1은 그 프로세스가 속한 network namespace의 loopback입니다.

같은 호스트·네임스페이스

- BFF → loopback Gateway를 우선 검토
- Gateway key는 BFF 서버 안에 보관
- 브라우저는 same-origin BFF만 호출

다른 호스트·컨테이너

- 서로의 127.0.0.1은 같은 서버가 아님
- 사설 라우팅·방화벽·서비스 인증 필요
- 경로 문자열만으로 mount가 생기지 않음

배치 위치를 먼저 그린 뒤 주소·인증·mount 계약을 정한다.

설계 토론 ① 편리하지만 위험한 연결

“브라우저가 path와 Gateway key를 보내면 구현이 간단하지 않을까요?”

이 설계에서 서로 다른 위험 세 가지를 찾고, 대체 경로를 제시해 보세요.

키가 브라우저에 노출되고, 사용자 입력 path가 파일 접근 권한처럼 동작하며, 관리 API까지 중계할 위험이 있습니다. 서버측 key + pageld allowlist + method/path allowlist로 바꾸고 Agent의 OS/도구 격리를 별도로 검증합니다.

cwd는 출발점이지 울타리가 아니다

current working directory는 상대경로를 해석하는 기준입니다.

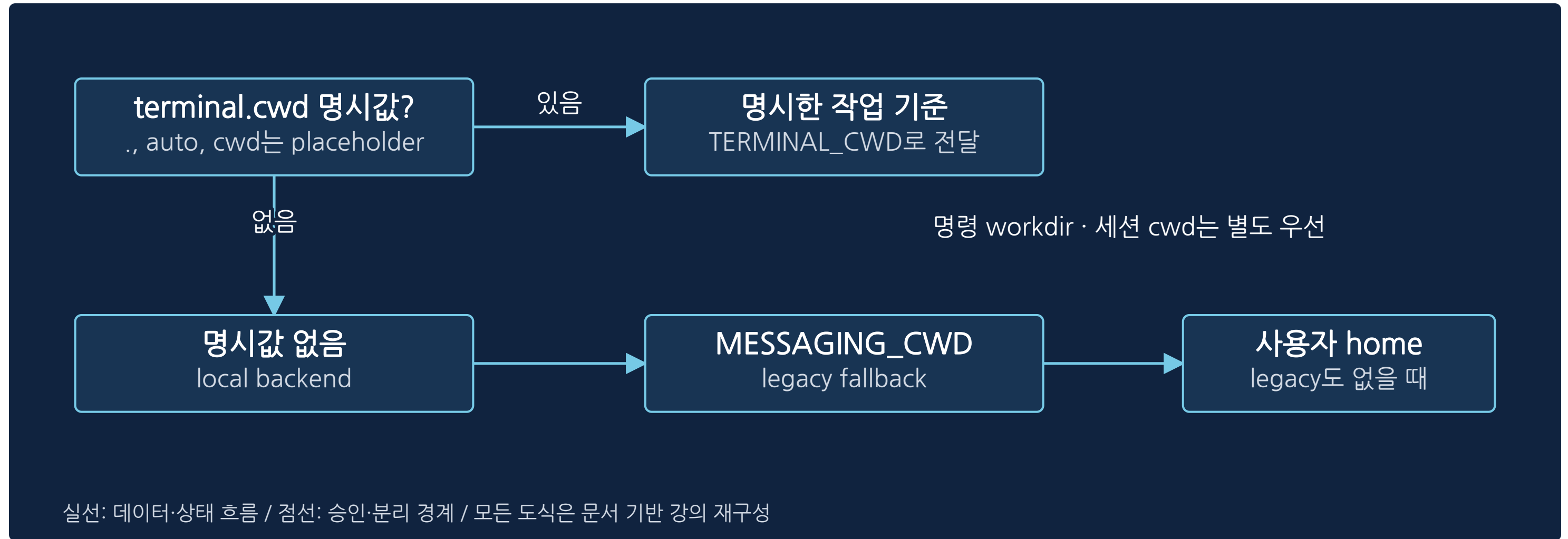
/home/ainsdev/wiki/pkm-articles

작업 루트 지정 ≠ HTTP document root 공개 ≠ 파일 접근 sandbox

local backend는 실행 사용자의 권한을 따르므로 Wiki 밖 절대경로도 별도 통제가 필요하다.

기본 cwd가 결정되는 경로

다음은 기술문서가 조사한 local Gateway 구현의 설명입니다.



명령 workdir·세션 cwd는 기본값보다 우선할 수 있다. 비로컬 backend는 mount까지 확인한다.

이름이 비슷해도 계약은 다르다

설정이 저장되었다고 런타임이 그 키를 읽는 것은 아닙니다.

이름	실제 의미 / 문서의 경계
terminal.cwd	도구의 기본 작업 기준 경로
WIKI_PATH	Wiki 탐색 관례; Gateway cwd API가 아님
MESSAGING_CWD	명시적 cwd가 없는 local 경로의 legacy fallback
request cwd / workdir / workspace_root	조사한 HTTP handler에 공개 적용 계약 없음
gateway.platforms.api_server.cwd	확인된 설정 계약 아님
/p/<profile>/...	프로필 라우팅; 임의 디렉터리 선택이 아님

설정 절차도 작은 변경으로 나눈다

다음은 원문의 승인 후 절차 예시입니다. 이 강의에서는 실행하지 않습니다.

```
hermes -p default config get terminal.cwd
hermes -p default config get terminal.backend

# 별도 변경 승인 후에만
hermes -p default config set terminal.cwd
/home/ainsdev/wiki/pkm-articles
hermes -p default config get terminal.cwd
```

원래 값과 영향을 받는 프로필·플랫폼을 먼저 기록한다.

config.yaml 전체나 terminal block을 직접 덮어쓰지 않는다.

config readback은 저장 확인이다. 실행 중 세션의 적용 증거는 아니다.

API bind와 secret은 별도 결정

cwd 변경 승인에 API 활성화·재시작 승인까지 들어 있지 않습니다.

```
# API enable/bind 변경까지 별도 승인한 경우만
hermes -p default config set gateway.api_server.enabled true
hermes -p default config set gateway.api_server.host 127.0.0.1
hermes -p default config set gateway.api_server.port 8642
```

API_SERVER_KEY는 운영자의 secret provisioning 경로로만 주입한다.

API_SERVER_* override와 terminal.cwd bridge의 우선순위를 혼동하지 않는다.

restart와 gateway run을 무조건 연속 실행하지 않는다. 실행 중 작업부터 확인한다.

적용 성공은 새 세션에서 확인한다

설정값 · 실제 도구 위치 · 프로젝트 지침 · 접근 권한은 각각 확인해야 합니다.

실행 경로

새 세션의 실제 tool cwd와 허용 자료 접근을 확인한다.

지침 발견

AGENTS 외 우선순위 파일·skip 설정·git-root 탐색·prompt 캐시를 고려한다.

권한 집행

쓰기 거부와 Wiki 밖 접근 제한을 실제 도구 경로마다 시험한다.

서비스 계약

승인 후 health → capabilities → 비민감 합성 API 시험으로 확인한다.

원문의 과거 terminal.cwd=. 관측을 현재값이나 무조건적인 rollback 값으로 재사용하지 않는다.

설계 토론 ② 설정 성공의 함정

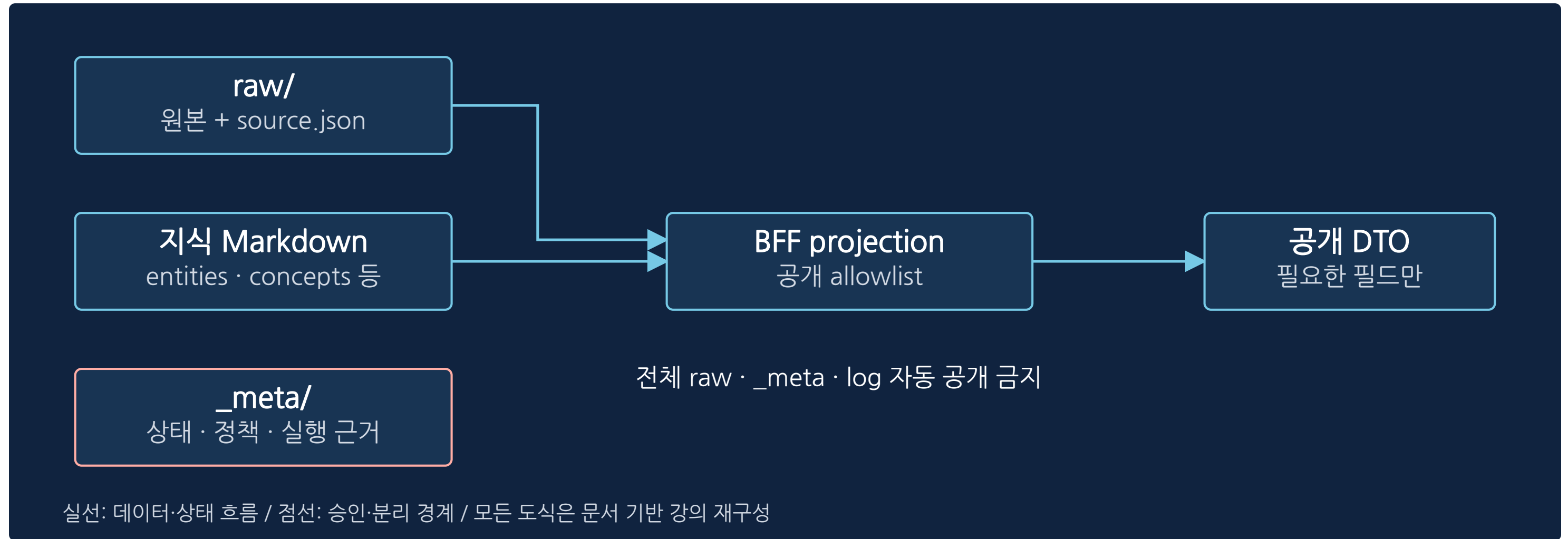
“임의 cwd 키를 저장했고 config get에도 나오니 적용 완료입니다.”

이 주장을 반박하는 데 필요한 확인 항목을 나열해 보세요.

실제 소비하는 설정 키와 handler를 먼저 확인합니다. 새 세션에서 tool cwd·프로젝트 지침을 관측하고, 비로컬 환경은 mount를 확인합니다. 마지막으로 OS/파일 도구의 접근 거부를 시험해야 합니다. 키 저장 성공은 지원 계약이나 sandbox 증거가 아닙니다.

파일 원장과 HTTP 응답은 다르다

저장소 구조를 URL 구조로 그대로 복사하지 않습니다.



원본·지식·상태·실행 근거를 분리하고, BFF는 필요한 지식·서지만 최소 투영한다.

원본 기록은 나중에 고쳐 맞추지 않는다

source.json의 `wiki_compiled:false`는 수집 당시의 정보입니다.

수집 시점

원본 HTML/PDF 바이트와 형식·버전·길이·hash를 보관한다.

컴파일 이후

완료 여부는 `compilation ledger + receipt + 실제 output`으로 확인한다.

잘못된 동기화

UI 배지를 맞추려고 불변 `source.json`을 수정하면 이력이 손상된다.

목록 API는 승인된 `metadata`를 읽는다. 목록 표시를 위해 PDF를 추출하지 않는다.

상태는 하나의 초록불이 아니다

서로 독립된 질문을 하나의 completed 배지로 합치지 않습니다.

상태 축	확인하는 질문	다른 의미로 확대 금지
원문 확보	파일을 보관했는가?	의미 분석 완료 아님
HTML 제공	공식 HTML이 제공되는가?	PDF까지 미확보란 뜻 아님
컴파일	읽기·생성은 어느 단계인가?	모델 종료 ≠ 게시 완료
게시 거래	committed인가?	실물 hash 검사를 대체 못함
인간 검토	human_reviewed 근거가 있는가?	자동 검사는 인간 검토 아님
API 실행	run이 종료되었는가?	논문 주장·업무 성공 보장 아님

페이지의 provenance를 화면에 남긴다

Provenance는 결과가 어떤 입력·정책·작업에서 왔는지 추적하는 정보입니다.

문서의 정체성

- schema / type / title / summary
- revision / transaction_id
- policy_revision / prompt_revision

검토의 범위

- read_scope / unread_scope
- status: draft / reviewed / superseded
- review_state: unreviewed / needs_review / human_reviewed

published_draft는 compilation 상태다. 페이지 status의 enum으로 바꿔 넣지 않는다.

DTO: 필요한 필드만 공개한다

아래는 제안된 BFF 응답의 축약 예시이며 실제 응답이 아닙니다.

```
{
  "schema": "pkm-web-page/v1",
  "page_id": "entity-arxiv-2609.30830v1",
  "revision": 2,
  "status": "draft",
  "review_state": "unreviewed",
  "content_sha256": "<실제 hex SHA-256>",
  "snapshot_id": "<committed snapshot 식별자>"
}
```

page_id는 서버가 생성·검증하는 식별자다. 파일 path가 아니다.

목록에는 본문·claim 상세·내부 evidence 경로를 생략한다.

pkm-web-page/v1과 snapshot_id는 Hermes의 기존 필드가 아니다.

ID를 파일로 바꾸는 순간이 경계다

허용된 ID라도 실제로 여는 파일까지 안전해야 합니다.

입력 제한

pageld → 승인된 Markdown 매핑을 사용한다. NUL·절대경로...혼합 구분자·중복 decoding·symlink를 거부한다.

열기까지의 일관성

canonical path 확인만으로는 검사와 사용 사이의 교체(TOCTOU)를 막지 못한다. 안전한 열기·snapshot·권한을 결합한다.

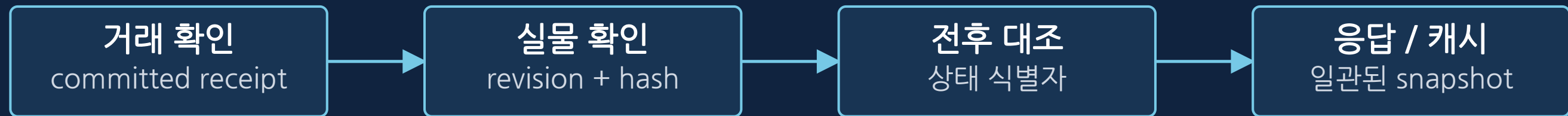
링크 해석

arxiv-2609.30830v1 뒤에 .md를 덧붙인다. 점 이후 문자열을 확장자로 오인해 치환하지 않는다.

서버 절대경로는 실제 웹 DTO에서 비공개다. 이 강의의 로컬 경로 표기는 설명용이다.

캐시도 검증한 snapshot만 기억한다

오래된 값보다 더 위험한 것은, 서로 다른 시점의 값을 한 페이지로 합치는 것입니다.



중간 상태가 바뀜 → 제한 재시도 또는 snapshot_conflict

실선: 데이터·상태 흐름 / 점선: 승인·분리 경계 / 모든 도식은 문서 기반 강의 재구성

충돌은 제한 재시도 또는 snapshot_conflict · process-local cache는 정식 원장이 아니다.

출처 링크도 범위를 말해야 한다

링크가 있다는 사실과 내용을 검토했다는 사실은 다릅니다.

HTML 근거

- 정확한 버전 source_url
- 실제 존재하는 anchor
- 불명확하면 “세부 위치 미확인”

승인된 PDF 텍스트 근거

- source.pdf#page=N
- 물리적 1-based 페이지
- 그림·불명료 수식·레이아웃 미검토 유지

HTML에 없는 PDF 페이지 번호를 만들지 않는다. 원문 링크 열기는 도표 검토 완료가 아니다.

같은 HTTP라도 두 종류의 API다

기존 Hermes API와 제안 BFF endpoint를 분리해서 읽습니다.

기존 Hermes · 문서/소스 조사

- /v1/chat/completions
- /v1/responses · /v1/runs
- 서버 간 Bearer 인증

제안 BFF · 아직 미구현

- /api/wiki/pages · /api/wiki/sources
- /api/agent/runs
- 사용자 인증 + 소유권 + 최소 응답

BFF endpoint가 문서에 있어도 현재 서비스에서 호출할 수 있다는 뜻은 아니다.

세 가지 실행 surface를 구분한다

같은 이름의 필드가 모든 endpoint에서 같은 의미를 갖지 않습니다.

surface	주 입력·이력	주 용도
Chat Completions	messages / stream X-Hermes-Session-Id header	범용 OpenAI 형식 client
Responses	input / instructions / store previous_response_id 또는 conversation	서버측 response chain
Runs	input / body session_id conversation_history / previous_response_id	비동기 실행·진행·중지 UI

최종 채택은 설치 capabilities와 실제 통합 시험 후 결정한다.

Chat 세션: header를 읽어야 한다

body session_id를 넣었다고 Chat의 대화가 선택되는 것은 아닙니다.

```
X-Hermes-Session-Id: <BFF 소유 transcript ID>

{
  "model": "hermes-agent",
  "messages": [
    {"role": "user", "content": "기존 노트의 검토 상태는?"}
  ],
  "stream": true
}
```

예시·미전송. Chat의 body session_id / conversation_id / user를 ACL로 쓰지 않는다.

header가 없으면 첫 user content와 system prompt에서 ID를 파생하는 설치 구현에 주의한다.

X-Hermes-Session-Key는 장기 기억 scope다. transcript ID나 인증 identity와 다르다.

Responses: 이력 연결을 명시한다

응답 체인과 conversation은 같은 요청에 동시에 지정하지 않습니다.

입력 계약

messages가 아니라 input, instructions 등을 사용한다. conversation_id가 아니라 conversation이다.

보존 계약

store와 이전 응답 연결의 저장·삭제 정책을 검토한다. 다른 사용자의 response ID를 신뢰하지 않는다.

중복 실행 방지

stream에 나온 function_call/output이 이미 서버에서 실행된 결과라면 client가 다시 실행하지 않는다.

OpenAI 호환이라는 말은 모든 role·tool history·이력 방식이 완전히 같다는 보장이 아니다.

Runs: 접수와 결과를 분리한다

202와 run ID는 “작업을 추적할 수 있다”는 시작점입니다.

```
POST /v1/runs
{
  "input": "허용된 기존 지식으로 질문에 답해 주세요.",
  "session_id": "<BFF 소유 세션 ID>"
}

# 접수 shape 예시 · 실제 실행 결과 아님
{"run_id":"run_<example>","status":"started","replayed":false}
```

상태는 GET /v1/runs/{id}로 조회한다.

queued/running/waiting_for_approval/stopping과 terminal 상태를 나눈다.

completed라도 partial/interrupted/error flags와 실제 결과를 함께 확인한다.

모델 이름과 실제 실행 모델은 다르다

model echo나 alias를 runtime lock의 증거로 쓰지 않습니다.

별칭과 선택 규칙

hermes-agent는 alias다. bare model·session override·model_routes 등의 적용 순서를 확인한다.

Fallback

model/provider/model_options를 보냈다는 사실만으로 fallback 금지가 보장되지 않는다.

업무 계약

고정 모델이 필요한 Wiki 컴파일은 별도 집행과 실제 runtime 검증을 거쳐야 한다.

이 강의는 모델을 호출하지 않는다. runtime 확인 절차를 설명한다.

권한은 method + path마다 좁힌다

일반 사용자에게 필요하지 않은 관리 surface는 공개하지 않습니다.

그룹	BFF 처리 제안
pages / sources / status	인증 + page ACL + 축약 projection
runs / events / stop	run owner + 정책 확인; 변경 요청 CSRF
approval / steer	필요 시 별도 scope; pending/running 상태 확인
/api/sessions/* / /api/jobs/*	관리 API 전체 프록시 금지; Cron 초기 차단
profile / tools / model / raw path	client의 임의 선택값을 그대로 upstream에 전달 금지

인증은 “누구인가”, 인가는 “이 객체에 이 작업을 해도 되는가”를 묻는다.

오류는 다음 행동으로 번역한다

HTTP 상태 하나를 “모델 오류”로 뭉뚱그리지 않습니다.

상태	의미 예	안전한 대응
400 / 413	입력 형식 / body 과다	요청 수정; 무한 retry 금지
401 / 403	인증 / origin·정책 거부	서버 credential·권한 확인; 키 출력 금지
404	객체·transport 없음	SSE 404면 run status도 확인
409	idempotency payload 충돌 등	같은 key의 원 요청 확인
429 / 503	동시성 제한 / drain	기존 run 존재·재시도 조건 먼저 확인

헤더 이후 stream이 실패할 수 있다. HTTP 200은 업무 완료 판정이 아니다.

스트림은 결과가 아니라 전달 경로다

SSE(Server-Sent Events)는 진행 이벤트를 서버에서 client로 전달합니다.

연결의 상태

- 연결됨 / 끊김 / 다시 연결 중
- 브라우저가 관측하는 transport
- 단절되어도 run은 계속될 수 있음

실행의 상태

- running / completed / failed 등
- Gateway의 작업 lifecycle
- GET run status로 재확인

재연결 ≠ 재실행 · 화면 새로고침에 새 run을 자동 생성하지 않는다.

SSE의 분기 위치는 surface마다 다르다

“event는 언제나 같은 곳에 있다”는 parser가 호환성 오류를 만듭니다.

surface	분기 위치	주의
Chat	data JSON + named event: hermes.tool.progress	[DONE]와 finish flags/usage 확인
Responses	response.* event family	이미 실행된 tool 결과 재실행 금지
Runs	data JSON 내부의 event 필드	run.completed/failed/cancelled/interrupted 구분

하나의 범용 parser에 endpoint별 event adapter와 의미 검증을 결합한다.

HTTP chunk는 메시지 경계가 아니다

한글 한 글자도, JSON 하나도 여러 chunk에 나뉠 수 있습니다.

```
data: {"event":"message.delta",  
data: "run_id":"run_<example>","delta":"한글"}
```

```
: keepalive
```

아래는 처리 단계 개념도

```
bytes → UTF-8 streaming decode → SSE frame → JSON →  
reducer
```

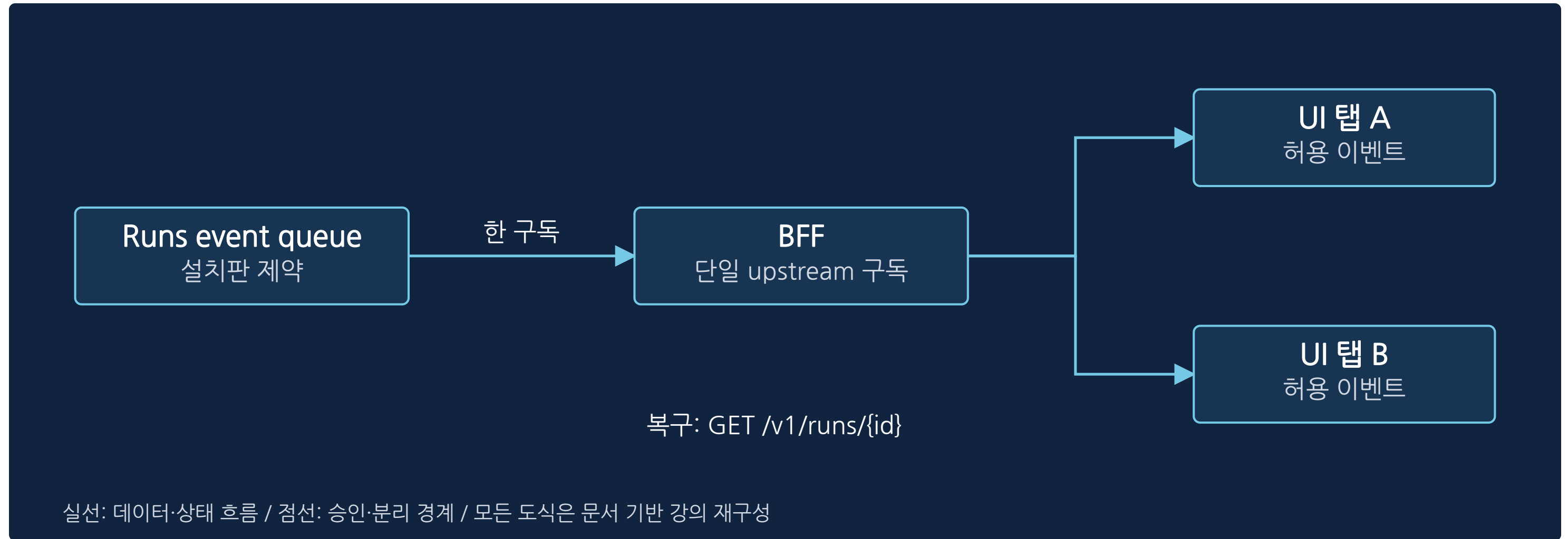
강의용 multi-line data 예시. data 값을 줄바꿈으로 합치면 유효한 JSON이 된다.

blank line, CRLF, multi-line data, comment를 규칙대로 처리한다.

unknown event는 안전하게 처리하되 알려진 종료·실패를 놓치지 않는다.

복수 구독자가 이벤트를 나눠 가져간다면?

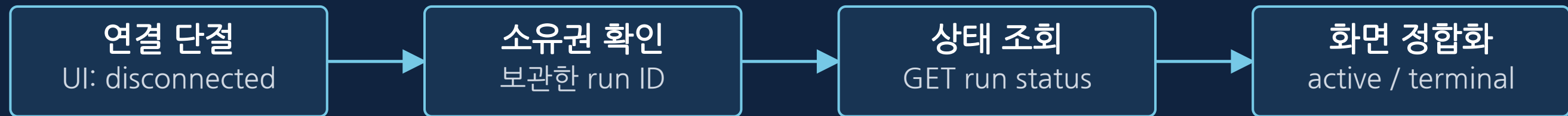
조사한 설치판 Runs는 단일 queue를 소비하며 연결 종료 시 transport를 삭제합니다.



BFF의 단일 upstream 구독 + 자체 fan-out + GET run status reconciliation을 권고한다.

재접속의 기준은 현재 run snapshot

끊어진 델타를 상상으로 복원하지 않습니다.



최종 snapshot은 임시 delta와 분리 · 자동 재실행 금지

실선: 데이터·상태 흐름 / 점선: 승인·분리 경계 / 모든 도식은 문서 기반 강의 재구성

Last-Event-ID 무손실 replay를 가정하지 않는다. 최종 snapshot과 임시 delta의 중복을 제거한다.

Idempotency는 같은 요청의 재전송을 막는다

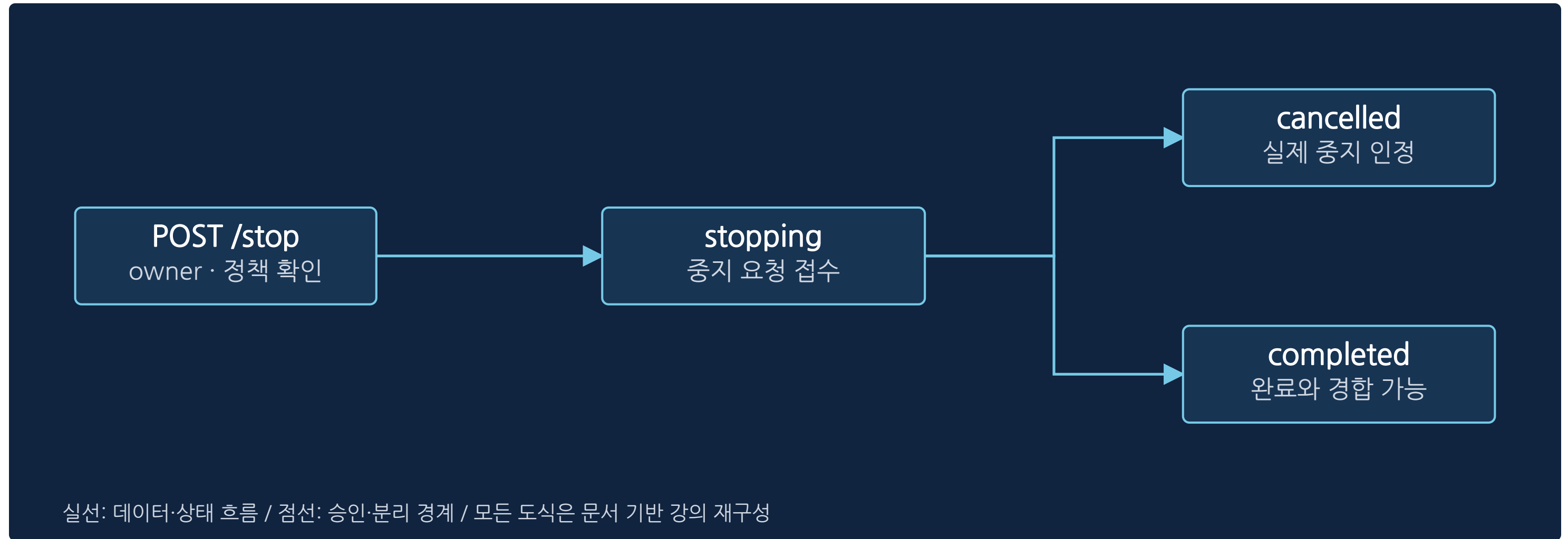
전송 응답이 유실되어도, 같은 일을 두 번 시작하지 않도록 합니다.

입력 조합	Runs 계약	해석
같은 key + 같은 payload	원 run의 202 replay	새 작업을 실행한다는 뜻 아님
같은 key + 다른 payload	409 conflict	질문을 고치면 새 key 필요
key 저장소 DB 실패	in-memory fallback 가능	durable capability 확인
Chat/Responses의 300초 캐시	별도 구현	Runs의 scoped durable 계약과 같지 않음

key: visible ASCII 1-255자 · terminal 기본 보존 24시간 · SSE replay/실행 재개와 별개

중지 요청은 취소 완료가 아니다

브라우저 연결 취소, 에이전트 중지, 이미 생긴 효과의 복구는 서로 다릅니다.



AbortController.abort()는 연결만 닫는다. stop은 이미 실행된 파일·외부 효과를 rollback하지 않는다.

steer와 approval도 접수 이후를 확인한다

버튼 클릭이나 200 응답만으로 사용자의 의도가 실행되었다고 말하지 않습니다.

steer

- running 상태의 지시 큐 등록
- 200은 큐 접수; 실제 소비와 다름
- pending_steer 등 미소비 상태 처리

tool approval

- 서버의 pending choices만 표시
- choice + 선택적 request_id 전달
- stale 요청 재사용·always 자동 선택 금지

승인 카드가 사라짐 ≠ 승인 성공 · 거부·만료·다른 화면 처리·연결 단절을 구분

설계 토론 ③ 새로고침 뒤 SSE가 404다

“오류가 났으니 새 run을 만들자”는 안전한 복구일까요?

run은 이미 생성되었고 응답 일부를 보았습니다. 어떤 순서로 복구해야 할까요?

BFF에 보관한 run ID와 소유권을 확인하고 GET /v1/runs/{id}로 현재 상태를 조회합니다. 실행 중이면 상태 추적을 이어가고, 종료되었으면 최종 결과로 UI를 정합화합니다. transport 404만으로 실패·취소를 단정하거나 새 작업을 자동 생성하지 않습니다. 사용자의 명시적 재실행과 전송 retry를 구분합니다.

기술 스택은 현재와 제안을 나눈다

제안 라이브러리를 설치된 구성요소로 설명하지 않습니다.

당시 조사한 Hermes

- API Server: aiohttp adapter
- asyncio + executor의 AI Agent
- 당시 manifest 0.21.1 / Python 범위 $\geq 3.11, < 3.14$

신규 앱 권고 · 미구현

- React + TypeScript / Next.js Node BFF
- Zod 등 DTO 검증 / 안전한 Markdown
- Vitest·Testing Library·Playwright 시험

Hermes API Server를 FastAPI라고 부르지 않는다. 권고 package version은 구현 시 다시 고정한다.

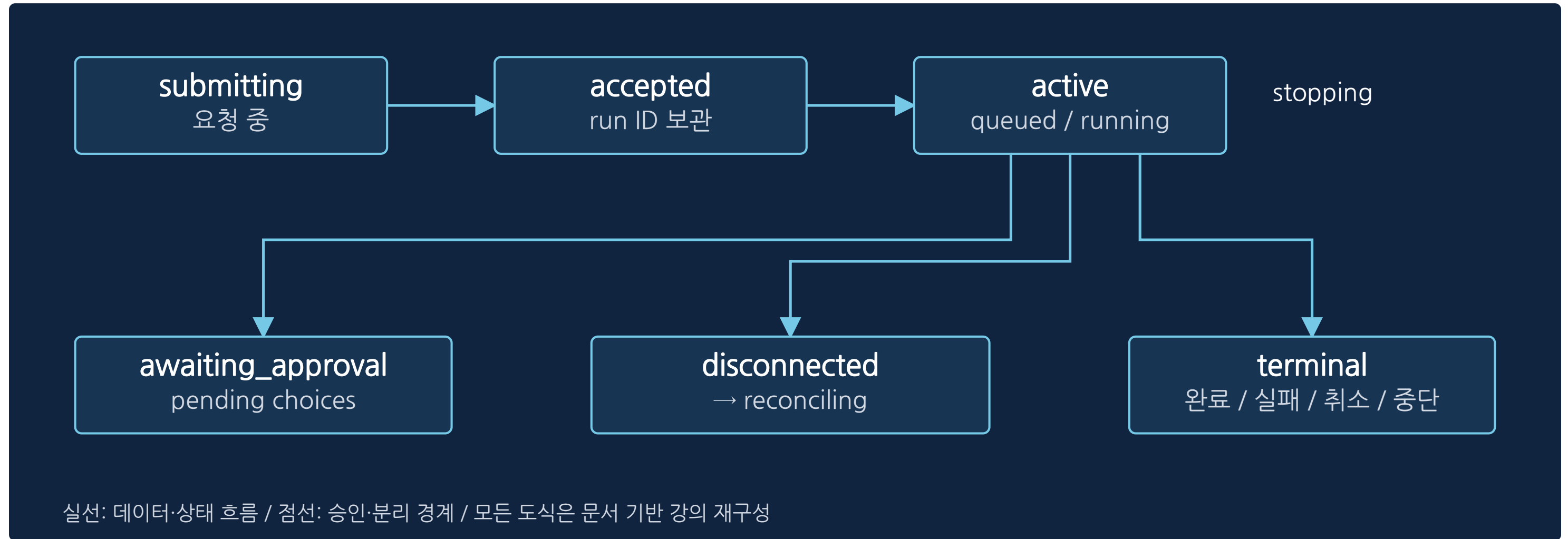
화면은 사용자의 질문 순서로

자료 찾기 → 노트 읽기 → 근거 확인 → 필요한 경우 질의

route	역할	권한 경계
/library	서지·보관 형식·노트 유무	read-only MVP
/knowledge/:pageId	본문·revision·검토 범위	page ACL / 안전 렌더링
/topics	개념·비교 연결	기존 지식 탐색
/chat/:conversationId	선택 문서·답변·실행 상태	후속 실행 승인
/operations	축약 관측 상태·시각	소유자 전용; Cron 수정 아님

UI 상태와 Gateway status를 함께 보관한다

disconnected는 UI의 관측이지 Gateway의 terminal 상태가 아닙니다.



commentary·tool·delta·final을 분리한다. raw tool 결과·비공개 추론을 무조건 노출하지 않는다.

비어 있음과 실패도 가르쳐야 할 정보다

아직 없는 노트, 권한 거부, 갱신 충돌을 같은 빈 화면으로 처리하지 않습니다.

상황	화면 표현	조작
필터 결과 없음	현재 필터에 맞는 자료 없음	필터 해제
지식 노트 미작성	원문 보관과 노트 없음 별도 표시	허용된 자료 탐색
snapshot 충돌	갱신 중 / 현재본 확인 불가	제한 재시도
run 중지 요청 중	stopping 표시	중복 요청 비활성
관측 정보 없음	미관측 / 확인되지 않음	0이나 성공으로 채우지 않음

읽을 수 있고 조작할 수 있어야 한다

토큰과 접근성 목표는 실제 화면 시험 전에는 통과 기록이 아닙니다.

시각 언어

원문 Civic Navy·차가운 중립색·sans를 사용한다. 상태는 색과 문자를 함께 보여준다.

키보드와 한국어

label·focus·dialog 복귀·IME 조합 중 제출 금지를 확인한다. 긴 ID와 한글 줄바꿈을 시험한다.

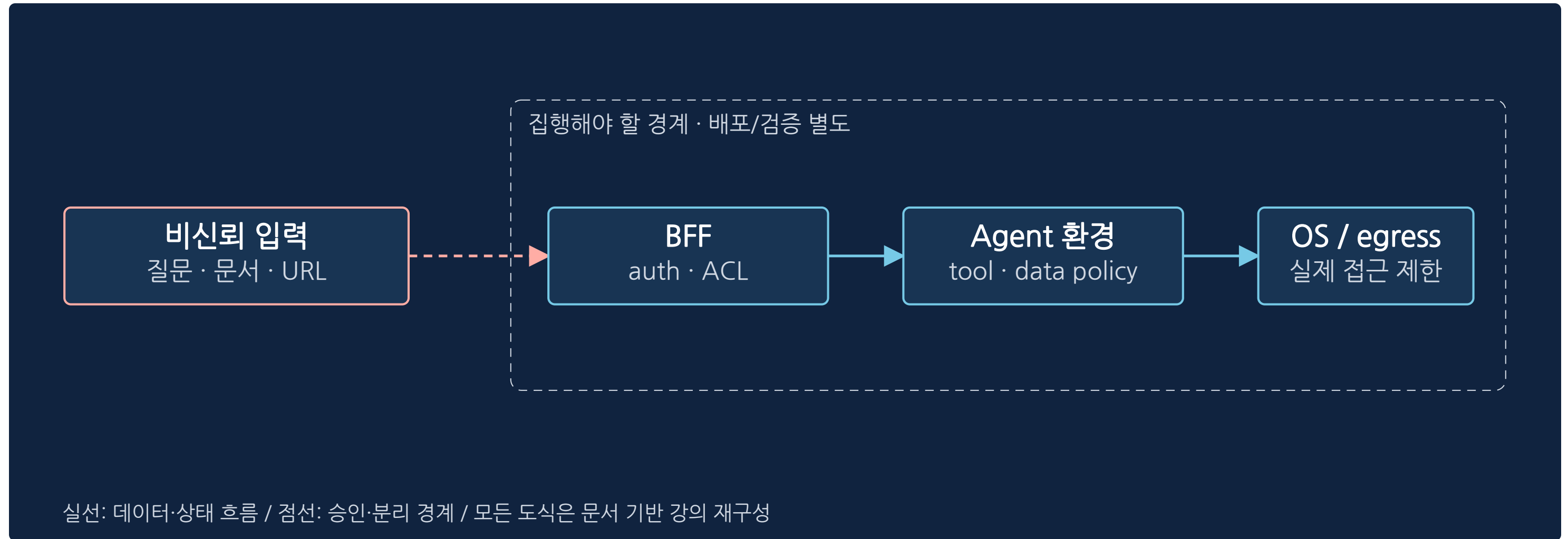
성능 증거

원문의 field 목표: p75 LCP < 2.5s, INP < 200ms, CLS < 0.1. baseline은 없으며 lab 결과를 field 통과로 바꾸지 않는다.

375/768/1280/1440px의 화면·오류·단절 상태까지 검증한다. 이 수치는 제안 앱의 목표다.

질문도 문서도 비신뢰 입력이다

자료 안의 지시는 실행 권한이 아닙니다.



보호 대상: Wiki·host 파일·자격증명·대화·승인 이력 · 프롬프트만으로 경계를 세우지 않는다.

Gateway key는 사용자 권한이 아니다

서버 간 credential과 브라우저 사용자의 identity를 분리합니다.

브라우저 ↔ BFF

- 인증된 single-owner 세션
- HttpOnly / Secure / SameSite
- 변경 요청의 CSRF·Origin 검사

BFF ↔ Gateway

- API_SERVER_KEY 서버측 보관
- run/session owner 매 요청 확인
- method/path allowlist와 프로필 경계

NEXT_PUBLIC_* · localStorage · URL query · 브라우저 번들/콘솔에 실제 key를 넣지 않는다.

격리는 실제 거부로 확인한다

“읽기 전용으로 행동하세요”와 “쓰기 시스템 호출이 거부됩니다”는 다릅니다.

파일시스템

최소 read-only mount/OS 권한을 사용한다. host home·credential·SSH key·Docker socket은 노출하지 않는다.

모든 도구 경로

terminal만 컨테이너에 있어도 다른 파일 도구가 host를 읽을 수 있다. 실제 실행 위치를 각각 시험한다.

전송과 자료 형식

container가 egress를 자동 차단하지 않는다. 도구 allowlist와 HTML/PDF 등 자료 형식의 허용 범위도 함께 검사한다.

프로필·cwd·AGENTS는 sandbox가 아니다. read_file이라는 이름도 비분석 도구를 뜻하지 않는다.

원본 보존과 안전한 표시를 양립시킨다

sanitize는 렌더링 경계의 일이지 불변 원본을 덮어쓸 이유가 아닙니다.

표시 계층에서 한다

- Markdown raw HTML 기본 off
- URL scheme 제한·sanitize·CSP 검토
- MVP에서는 공식 버전 고정 원문 링크

자동으로 하지 않는다

- source.html 원본 덮어쓰기
- 앱 origin에서 raw HTML 그대로 실행
- PDF preview 명목의 추출·OCR·렌더링

XSS 방어와 원본 불변성은 서로 대체 관계가 아니다.

SSE 프록시와 로그도 계약의 일부다

연결이 살아 있는 것과 데이터가 즉시 전달되는 것은 다릅니다.

전송

- text/event-stream · buffering/변환/캐시 금지
- 실제 keepalive·idle timeout 시험
- CORS는 인증·방화벽 대체가 아님

로그

- request/run ID·시간·HTTP/run status
- 허용 문서 revision/hash·정책 결과
- key·cookie·대화 원문·raw tool 결과 제외

당시 공식 문서 10초 / 로컬 Runs wait timeout 30초의 차이: 설치판을 실제 측정해야 한다.

장애 대응도 실패를 숨기지 않아야 한다

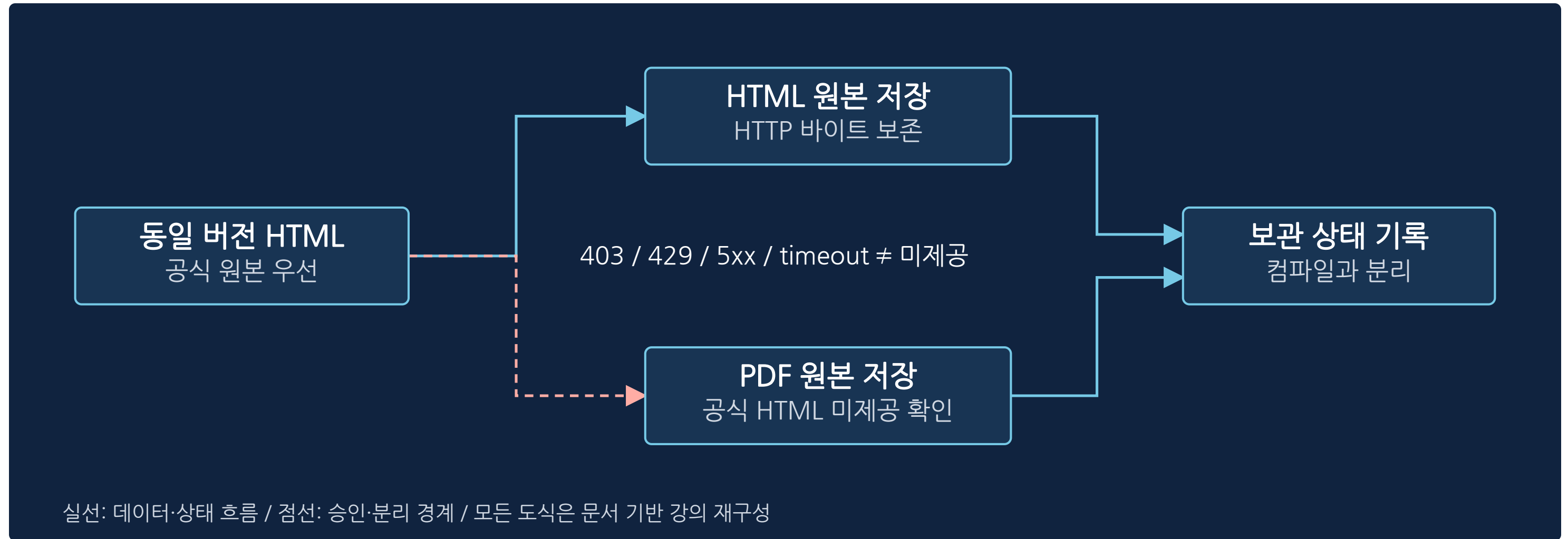
관측한 신호에서 출발하고, 권한 우회로 해결하지 않습니다.

관측 신호	먼저 확인	하지 않을 대응
health 성공, 질의 실패	readiness·auth·model·tool	전체 정상 선언
SSE 단절	run 상태·buffering·timeout	새 run 자동 생성
stopping 장기 지속	실제 executor·도구 상태	cancelled로 숨김
interrupted	shutdown/restart 이력	성공 승격·효과 자동 재실행
snapshot 충돌 / lock 존재	receipt·hash·lock owner	partial 캐시·오래된 lock 자동 삭제

앱 배포 rollback과 Wiki 원본/상태 rollback은 다른 작업이다.

수집 성공은 지식 생성 완료가 아니다

기존 Cron은 원본 저장 전용이며 웹 앱의 배포·질의와 분리됩니다.



403/429/5xx/timeout은 HTML 미제공이 아니다. 수집 완료를 자동 컴파일 trigger로 연결하지 않는다.

v3 예외는 좁은 자료 경로다

별도 승인된 로컬 PDF 내장 텍스트만 메모리에서 읽는 수동 경로입니다.

승인 범위 안의 수동 경로

- 표준 pypdf · embedded text only
- 정확한 입력·단계·모델·무결성 gate
- 물리적 페이지 인용·미독 범위 기록

계속 금지 / 자동 승계 없음

- PDF 바이너리·이미지 전송
- OCR·렌더링·지속 추출본
- 웹 자유 질의로 예외 권한 확대

codex-lb / gpt-6-astra / xhigh · no-fallback · 비용 미관측 null · no_cost_cap도 승인 면제가 아니다.

두 종류의 승인은 서로 대체할 수 없다

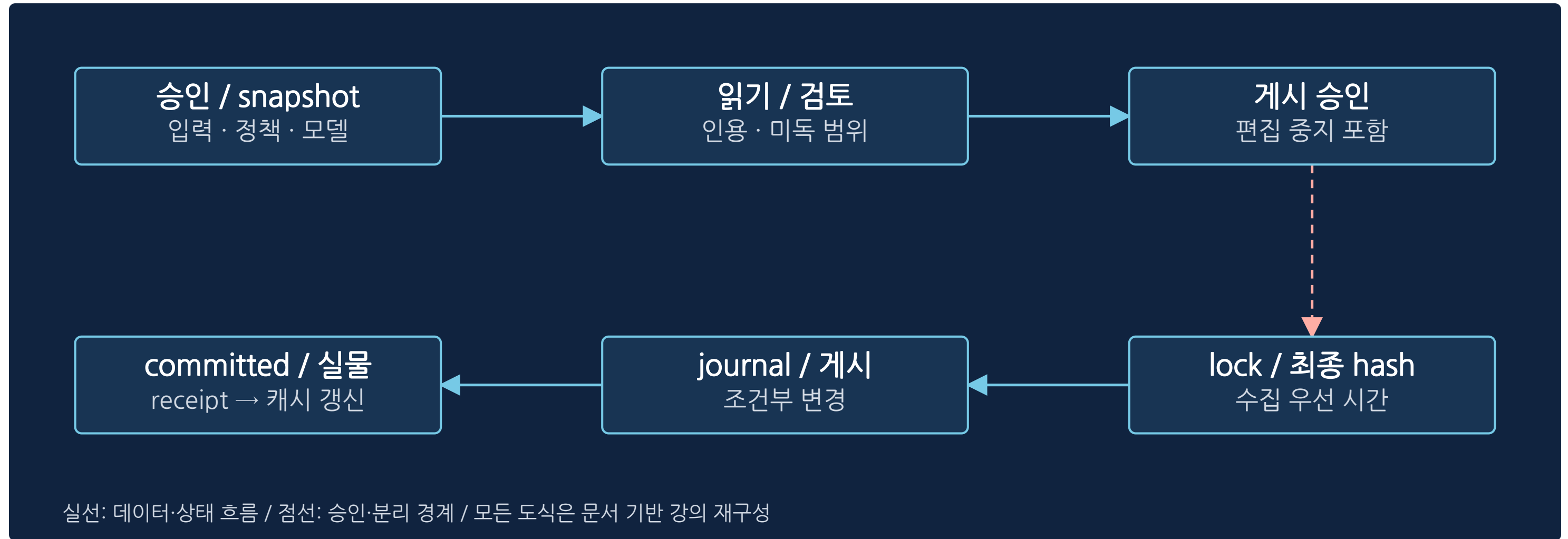
특정 도구 실행 승인과 특정 지식 작업의 게시 승인은 질문이 다릅니다.

계층	승인의 대상	대체하지 못하는 것
Hermes tool approval	pending 도구 동작의 허용/거부	논문·자료·모델·게시 단계 승인
Wiki 정책·실행 승인	정확한 입력 snapshot과 처리 범위	원문 무결성·실제 모델 검증
게시·편집 중지 승인	정확한 검증 결과의 조건부 게시	lock·최종 hash·journal·receipt 확인

일반 채팅 완료를 entities/ 수정 허가로 읽지 않는다. 초기 UI에는 compile 버튼을 두지 않는다.

게시 완료에는 파일 이상의 증거가 필요하다

여러 파일의 게시를 OS가 한 번에 원자적으로 보여준다고 가정하지 않습니다.



write-ahead journal은 변경 전 기록, receipt는 게시 결과 증거다. committed와 실물을 함께 대조한다.

동시성 제한은 서로 다른 문제를 푼다

run 개수 제한 하나로 Wiki의 다중 파일 일관성이 해결되지 않습니다.

경계	막으려는 충돌	확인 대상
BFF 세션 단일 writer	같은 대화의 동시 turn	session ownership·직렬화
API concurrent-run 제한	실행 자원·남용	429·유한 재시도
collection.lock	수집자와 게시자의 동시 쓰기	owner·수집 우선 시간
최종 hash·journal·receipt	사용자 수정·부분 게시·복구 충돌	조건부 갱신·exact-repeat no-op

무한 대기·고아 lock 자동 탈취·과거 snapshot 전체 복원을 하지 않는다.

설계 토론 ④ 어떤 배지가 정직한가?

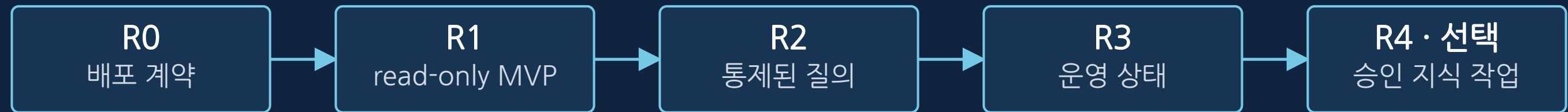
PDF 보관 완료, run completed, 하지만 receipt가 아직 committed가 아닙니다.

원문·실행·게시·검토 상태를 각각 어떻게 표시해야 할까요?

원문은 “PDF 보관”, 실행은 관측된 run 상태와 결과 flags를 표시합니다. 게시 거래는 “미해결/완료 확인 불가”로 남기고 최신 완료본으로 캐시하지 않습니다. 인간 검토 근거가 없으면 unreviewed를 유지합니다. completed 하나로 모두 성공 처리하지 않습니다.

작게 만들되 검증 경계는 줄이지 않는다

R0-R4는 제안 서비스의 구현 단계이며 Wiki의 P3-P6 승인과 다릅니다.



각 단계의 수용 기준을 검증 · R4와 Wiki P3-P6 실행 승인은 별개

실선: 데이터·상태 흐름 / 점선: 승인·분리 경계 / 모든 도식은 문서 기반 강의 재구성

R4는 선택 단계다. 로드맵 존재를 배포·컴파일·자동화 실행 허가로 해석하지 않는다.

실습 ① read-only MVP의 계약을 그리기

코드 실행 없이도 자료 노출과 일관성 문제를 설계할 수 있습니다.

합성 pageld 하나로 목록/본문 DTO와 실패 응답을 설계하세요. 인증 없음, 잘못된 ID, hash 불일치, 게시 중 충돌, 악성 Markdown을 포함합니다.

인증·page ACL → 서버 pageld 매핑 → committed receipt와 hash 대조 → 안전 렌더링 순서가 필요합니다. 목록은 최소 필드만, 충돌은 snapshot_conflict로 반환합니다. 실행 구현 시 원본/수집 state 불변과 실제 write 거부 시험을 수용 기준으로 둡니다. 합성 fixture 결과를 실제 서비스 통과라 부르지 않습니다.

실습 ② 끊겨도 중복 실행하지 않는 채팅

이벤트 전달 실패를 업무 재실행으로 바꾸지 않는 reducer를 설계하세요.

① 요청 timeout ② 한글 byte 분할 ③ SSE 단절 ④ stopping 중 completed 도착을 넣고, 각 상황에서 UI와 BFF의 행동을 적어 보세요.

① 생성 여부를 먼저 확인하고 같은 요청은 같은 idempotency key로 retry합니다. ② streaming UTF-8 decode 후 프레임을 조립합니다. ③ 보관한 run ID의 status로 정합화하며 자동 새 run은 금지합니다. ④ 실제 completed와 결과 flags를 반영합니다. stop 접수가 취소 완료를 보장하지는 않습니다.

테스트 이름보다 실행 범위를 기록한다

합성 시험, 실제 통합 시험, 배포 검증은 서로 다른 증거입니다.

계층	정적·합성 시험	실제 환경에서의 시험
Data	ID·enum·hash·경로 fixture	snapshot 충돌·read-only 권한
API / SSE	allowlist·ownership·프레임 reducer	capabilities·auth·proxy timeout
Security	악성 URL/Markdown/symlink	키 비노출·CSRF·격리
UX	상태 전이·IME·긴 텍스트	viewport·키보드·screen reader
Reliability	중복·취소·경합	restart·stop 경합·상태 복구

실행하지 않은 것은 not_run · fake provider 성공은 실제 모델 실행 성공이 아니다.

버전 차이를 숨기지 않는 개발

최신 공식 문서와 설치 checkout은 다른 시점을 가리킬 수 있습니다.

차이 사례	강의의 결론
keepalive 간격	공식 10초 / 로컬 30초 → 설치판과 proxy 실측
Runs event queue	transport 삭제 → 무손실 replay 가정 금지
CORS 정적 위험	브라우저 재현 전 확정 장애로 표현 금지
runtime / terminal	실제 capabilities와 응답으로 협상
turn lease	모든 호출 경로의 직렬화 검증과 구분

문서 기준 Hermes HEAD: 8d79c2ff57bba4b07e5b37ed90387b16541aef53 · 현재 runtime 재검증 아님

마지막 점검: 무엇을 안다고 말할 수 있나?

“그럴듯한 완료”보다 “관측한 범위의 완료”가 더 유용합니다.

원문 기술문서가 검증한 것

- 로컬 문서 링크·fence·JSON 구문
- Bash는 bash -n 구문 검사
- 당시 metadata·hash·설정 leaf 읽기

그 기록이 증명하지 않는 것

- live API·모델·서비스 기동
- BFF/frontend 배포·운영 준비
- 접근성·성능·논문 본문 분석

이 강의 HTML의 렌더링 검증도, 설명 대상 웹 서비스의 운영 검증은 아니다.

기억할 설계 원칙

모든 “완료”에는 무엇이 끝났는지 목적어를 붙입니다.

cwd는 위치, sandbox는 권한.

BFF는 최소 계약, API는 실행 계약.

연결과 실행, 생성과 게시, 자동 검사와 인간 검토를 분리한다.

다음 구현의 첫 단계: 배포·권한 계약 확정 → 모델 없는 read-only MVP

원문으로 돌아가는 방법

각 슬라이드의 “근거”에서 사용한 원문 구간과 줄 번호를 확인할 수 있습니다.

입력 범위

기술 Markdown 12개와 verification.json을 읽고 강의 흐름으로 재구성했다. 원문 논문 HTML/PDF는 읽지 않았다.

인용 방식

로컬 기술문서의 파일명·줄 범위를 붙였다. 원문이 인용한 외부 소스를 이번 강의에서 독립 재검증한 것으로 표시하지 않는다.

이동과 보관

HTML에는 인용 구간·폰트·SVG·코드가 내장되어 있다. 원문 전체 링크는 이 디렉터리 구조가 유지되어야 열린다.